10. Lambda Expressions — Quick Notes
Definition

Lambda Expressions are: Anonymous Functions

Meaning:

- No function name
- No separate class needed

Added in: Java 8

Functional Interface
Definition

Interface with: exactly ONE abstract method

Can have:

- default methods
- static methods

Example:

- Predicate
- Runnable
- Comparable

Predicate Interface
Example Functional Interface

```
interface Predicate<T> {
        boolean test(T t);
}
```

- Takes input
- Returns boolean

Why Lambda Expressions?

Without Lambda:

- Create class
- Implement interface
- Create object

Too much boilerplate code.

Old Way (Without Lambda)

```
class MyPredicate implements Predicate<Integer> {

   public boolean test(Integer x) {
      return x % 2 == 0;
   }
}
```

Usage: Predicate<Integer> p = new MyPredicate();

Anonymous Class (Before Java 8)

```
Predicate<Integer> p =
   new Predicate<Integer>() {

      public boolean test(Integer x) {
         return x % 2 == 0;
      }
};
```

Better than separate class, but still lengthy.

Lambda Expression Syntax

parameter -> logic

Example: x -> x % 2 == 0

Meaning: return true if x is even

Lambda Example

```
Predicate<Integer> p = x -> x % 2 == 0;
```

This becomes implementation of: test() method internally.

Real Example

```
List<Integer> list = Arrays.asList(10,5,20,7,30);
list.forEach(x -> System.out.println(x));
```

Output: 10 \n 5 \n 20 \n 7 \n 30

Filtering Example

```
list.removeIf(x -> x % 2 == 0);
```

Removes: even numbers

Important Point

Lambda works ONLY with: Functional Interfaces

Because the compiler maps Lambda to single abstract method.

Common Functional Interfaces

Interface	Abstract Method
Predicate	test()
Runnable	run()
Comparable	compareTo()
Consumer	accept()

Lambda Advantages

Feature	Benefit
Concise code	Less boilerplate
No separate class	Cleaner code
Functional style	Works great with Streams
Easy filtering	Cleaner collection operations

Important Quick Points

Topic	Point
Lambda	Anonymous function
Added in	Java 8
Works with	Functional Interfaces
Functional Interface	One abstract method
Arrow Operator	`->`
Common use	Streams, removeIf(), sorting